



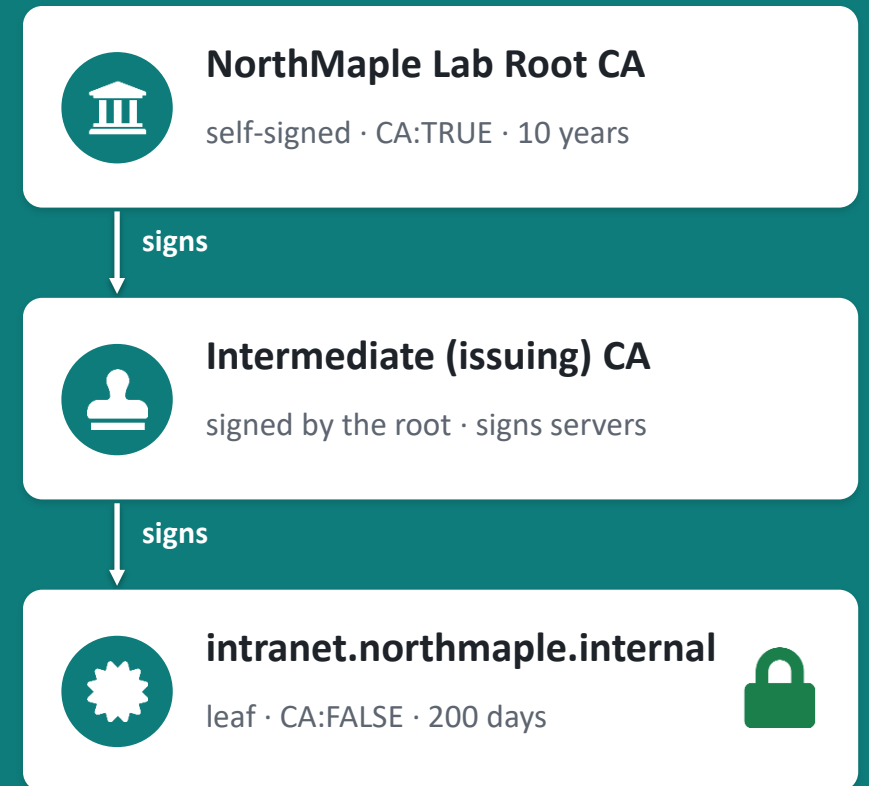
PKI Concepts

PKI (Public Key Infrastructure): digital certificates and Certificate Authorities, how a browser trusts a website, **HTTPS** (Hypertext Transfer Protocol Secure), **TLS** (Transport Layer Security) and **SSH** (Secure Shell), the certificate life cycle, and how cryptography is assessed for each application.

Cybersecurity AEC (Attestation d'études collégiales) · LEA.D8

Competency EZ03 — To apply access control measures

4 lessons · 60 minutes · 4 real-life examples





Before we start: sign for Session 15



Sign for Session 15

- Open **Acknowledgment_and_Agreement_e Sign.html** on the course platform.
- Sign for **Session 15**, download your signature record and submit it.
- Required at the beginning of **every** session (Legal Notice §16).



Educational purpose only

- This Material teaches **defensive** cybersecurity: how to protect systems.
- Nothing in it authorizes you to access, test or damage any real system (§2).



Using it means accepting it

- Reading, watching, downloading or using any part of the Material means you accept the whole **Legal Notice, Disclaimer, and Acceptable Use Policy**, Version 1.0 (§1, §16).



You are fully and solely responsible for your own actions, equipment, accounts, costs and time (§1).

Privacy: the signature log records your name, the session number, the date and a verification code; your drawn signature stays only in the record file you submit (§14).



Acceptable use and legal warning



You may

- ✓ Practice **only** inside your personal, isolated laboratory (LAB_01): VirtualBox, or your own free-tier **AWS** (Amazon Web Services) or Azure account.
- ✓ Practice on another system **only** with the explicit written permission of its owner, obtained **before** you start.
- ✓ Use snapshots and isolated virtual networks, so that a mistake stays inside your laboratory.



You must never

- ✗ Scan, probe, attack, intercept, monitor or access any system, network, Wi-Fi or account that is not yours.
- ✗ Create, install or release malicious software outside your isolated laboratory.
- ✗ Attempt password, credential or interception attacks against any real account or communication.
- ✗ Use the Material or its tools to break any law or policy.



Unauthorized computer access is a criminal offence. Canada, Criminal Code: s. 342.1 (unauthorized use of a computer), s. 342.2 (device to obtain unauthorized use of a computer), s. 430(1.1) (mischief in relation to computer data) — penalties can include imprisonment. Outside Canada, your own laws apply; college policies apply too. **If an activity seems to touch a system you do not own, stop and ask the instructor.**

Cloud accounts: respect the provider's acceptable-use policy; free tiers have limits; all costs are yours (§3).

Recall: Class 14 in five points



Symmetric encryption

One shared key encrypts and decrypts, for example **AES** (Advanced Encryption Standard). Fast; the hard part is sharing the key safely.



Asymmetric encryption

A **key pair**: the public key encrypts or verifies; the private key decrypts or signs (**RSA** (Rivest-Shamir-Adleman), **ECC** (Elliptic Curve Cryptography)).



Hash function

Any input becomes a fixed-length digest, for example **SHA-256** (Secure Hash Algorithm, 256-bit output); one changed bit changes the whole digest.



Salted hashes

Passwords are stored as **salted hashes**, never in clear text.



In transit and at rest

Cryptography protects data **in transit** (on the network) and **at rest** (on a disk).



Today's question: when a server sends me its public key, how do I know it is really the key of that server? The **PKI** (Public Key Infrastructure) answers it.



Today's objectives and time plan

At the end of this class, you will be able to:

- 1 Explain the problem a **PKI** (Public Key Infrastructure) solves and name its parts: certificate, **CA** (Certificate Authority), **RA** (Registration Authority), trust store, **CRL** (Certificate Revocation List) and **OCSP** (Online Certificate Status Protocol).
- 2 Read the main fields of an X.509 certificate and list the checks a browser runs before it trusts a website.
- 3 Explain how **TLS** (Transport Layer Security) uses a certificate in **HTTPS** (Hypertext Transfer Protocol Secure), and how **SSH** (Secure Shell) trusts host keys instead.
- 4 Describe the nine stages of the certificate life cycle, including renewal and revocation.
- 5 Assess whether a cryptographic system is adequate for one application.

Time plan — 60 minutes

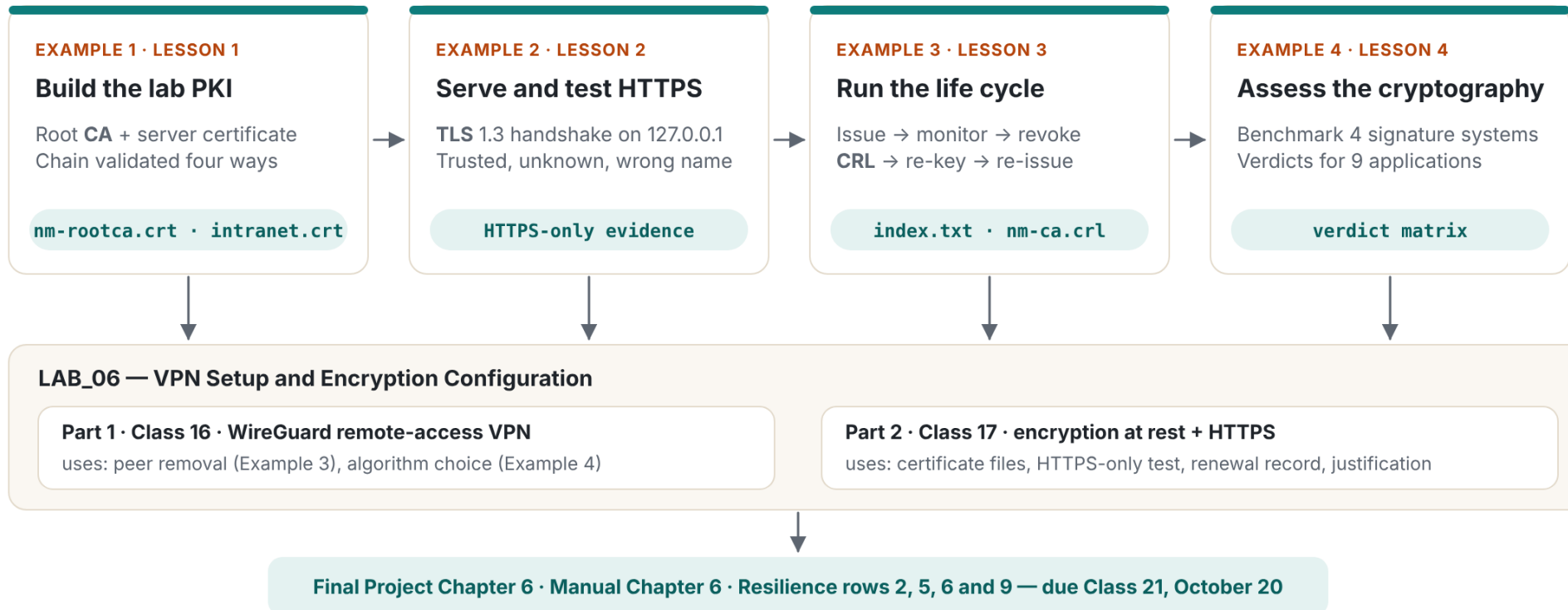
- | | |
|------|---|
| 0:00 | Recall and objectives
2 min |
| 0:02 | Lesson 1 — certificates and browser trust
18 min |
| 0:20 | Lesson 2 — HTTPS , TLS and SSH
14 min |
| 0:34 | Lesson 3 — certificate life cycle
12 min |
| 0:46 | Lesson 4 — assessing cryptography
11 min |
| 0:57 | LAB_06 preview and work time
3 min |



Class 15 at a glance

Four real-life NorthMaple examples form one chain: each produces what the next one — and LAB_06 — reuses

Class 15 — four real-life examples, one chain of deliverables



PKI = Public Key Infrastructure · CA = Certificate Authority · HTTPS = Hypertext Transfer Protocol Secure · TLS = Transport Layer Security · CRL = Certificate Revocation List
VPN = Virtual Private Network

Cyber Defense (420-D03-AB) · Class 15 · Cyber.SoHo

The problem a PKI solves



Accountant

NM-WS01



Attacker

on the path



Intranet server

intranet.northmaple.internal

←
attacker's public key

←
real public key

The workstation encrypts for the attacker, who reads or changes the data and forwards it. Neither side notices: this is a **MITM** (Man-in-the-Middle) attack.

A stronger algorithm does not help: the weakness is the missing **binding between an identity and a public key**.



What a PKI (Public Key Infrastructure) is

A **PKI** is the combination of roles, policies, procedures, software and hardware used to **create, distribute, store, validate and revoke** digital certificates.

Its single purpose: provide the identity ↔ public key binding in a form a computer checks **automatically, in a few milliseconds**, without asking the user anything.

Five key terms of a PKI



Key pair

Two related keys. A signature made with the private key is verified with the public key; data encrypted with the public key is decrypted with the private key.



Digital certificate

A signed statement that binds a public key to an identity (a name) for a defined period and for defined uses.



CA (Certificate Authority)

The entity that verifies an identity, then signs the certificate that binds it to a public key.



Trust store

The list of root certificates that an operating system or a browser accepts without further proof.



Revocation

Declaring a certificate invalid before the end of its validity period.

The X.509 v3 certificate (1/2)

The Internet profile of X.509 is **RFC** (Request for Comments) 5280 — these are the fields you read in the output of Example 1

Field	What it contains	Why the client needs it
Version	3 — the version that supports extensions	Tells the client how to read the structure
Serial Number	Unique among all certificates of the same CA (Certificate Authority)	Identifies the certificate in revocation lists
Signature Algorithm	<code>ecdsa-with-SHA256</code>	How to verify the signature of the issuer
Issuer	The DN (Distinguished Name) of the CA that signed	Finds the issuer certificate when the chain is built
Validity	Not Before and Not After dates	Rejected outside this time window
Subject	The DN of the owner: CN (Common Name), organization, country	Identifies the owner
Subject Public Key Info	The algorithm and the public key itself	The key the certificate vouches for
Subject Alternative Name	DNS (Domain Name System) names and IP (Internet Protocol) addresses	Compared with the host name in the address bar
Key Usage, Extended Key Usage	For example digitalSignature; serverAuth	Limits what the key may be used for
Basic Constraints	<code>CA:TRUE</code> or <code>CA:FALSE</code>	Only <code>CA:TRUE</code> may sign other certificates
CRL (Certificate Revocation List) Distribution Points, Authority Info Access	Addresses of the revocation list and of the issuer certificate	Revocation checks; download missing intermediates
Signature Value	The issuer's signature over all the fields above	Proves the issuer approved this exact content



The X.509 v3 certificate (2/2)



The signature value makes it trustworthy

- 1 The issuer hashes all the other fields and signs the hash with its **private key**.
- 2 Any client holding the issuer's **public key** recomputes the hash and verifies the signature.
- 3 Change one bit — a name, a date, a key — and the verification fails.



A certificate is **public**: holding a copy proves nothing. Identity is proven by signing fresh data with the matching **private key** (Lesson 2).



PEM (Privacy-Enhanced Mail)

`.pem .crt .cer`

Base64 text between BEGIN and END CERTIFICATE lines; the default of OpenSSL, Linux and pfSense.



DER (Distinguished Encoding Rules)

`.der .cer`

The same certificate in binary form.



PKCS (Public-Key Cryptography Standards) #12

`.pfx .p12`

Certificate + private key in one password-protected file (Windows Server, NM-FS01).

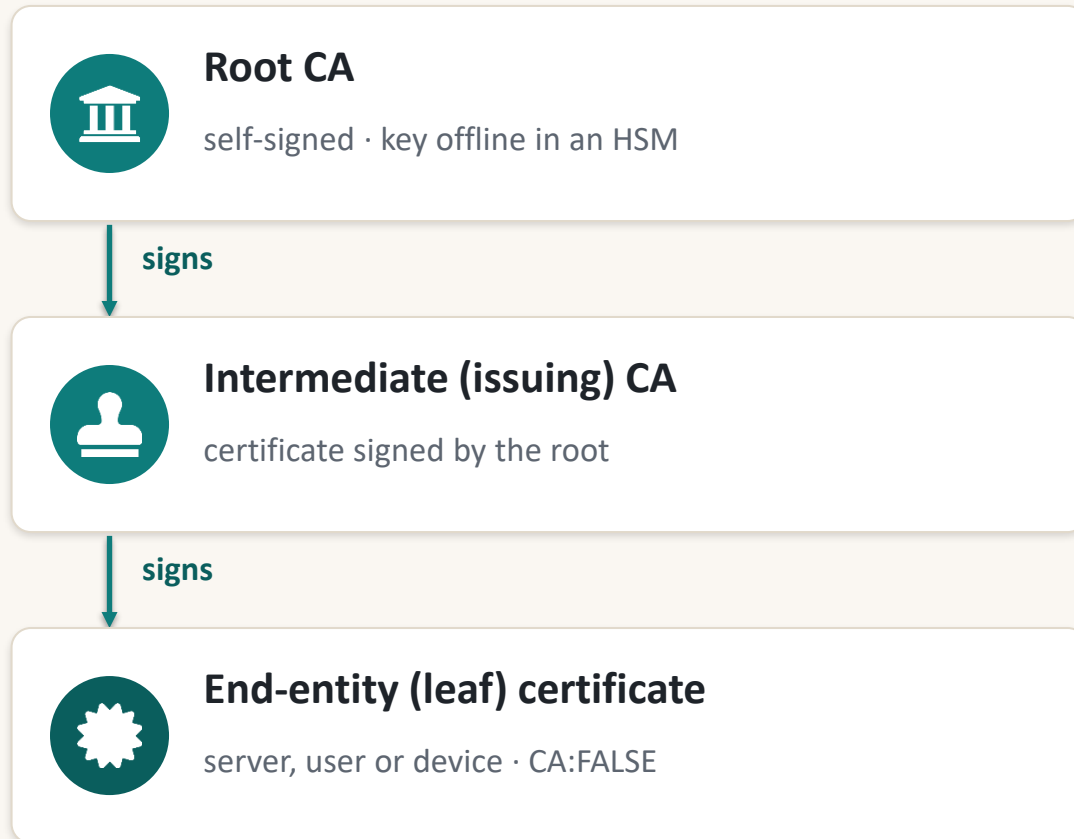


CSR (Certificate Signing Request)

`.csr`

Public key + requested names, signed with the requester's private key.

The hierarchy of Certificate Authorities



- **Root CA** (Certificate Authority) — Issuer and Subject are identical, because no higher authority exists. Trusted only because it is pre-installed in a trust store; signs only intermediate certificates and revocation lists.
- **Intermediate CA** — signs end-entity certificates every day. If its key is compromised, the root revokes that single intermediate; the root, already in millions of trust stores, is not replaced.
- **Leaf certificate** — its Basic Constraints extension says `CA:FALSE`: it may not sign other certificates.
- **RA** (Registration Authority) — an optional role that performs the identity checks for a **CA** before the **CA** signs.

Public CA, private CA and validation levels



Public CA (Certificate Authority)

- Trusted by browsers and operating systems worldwide — for example **Let's Encrypt**, a free, automated **CA** operated by the non-profit **ISRG** (Internet Security Research Group).
- Issues certificates **only** for names it can validate on the public Internet.



Private CA

- Run by an organization for its own internal names; trusted only by devices where its root was installed — the NorthMaple lab **CA** (Example 1), the pfSense Certificate Manager.
- `intranet.northmaple.internal: .internal` was reserved by **ICANN** (Internet Corporation for Assigned Names and Numbers) on July 29, 2024 for private networks; it is never delegated on the Internet.

DV (Domain Validation)

The requester proved control of the domain name, usually automatically.

OV (Organization Validation)

The legal identity of the organization was also checked.

EV (Extended Validation)

A stricter verification of the legal entity.

Browsers display the same padlock for the three levels; the level is visible only in the certificate details.

Trust stores and root programs

1



A CA (Certificate Authority) asks to be trusted

Independent audits of its practices.

2



A root program admits it

Mozilla **CA** Certificate Program, Microsoft Trusted Root Program, Chrome Root Store.

3



Its root enters the trust stores

Windows, macOS, most Linux distributions, Firefox, Chrome.

4



It stays only while compliant

CA/Browser Forum (Certification Authority/Browser Forum) **Baseline Requirements**; audit records shared through the **CCADB** (Common Certificate Authority Database).



2011: the DigiNotar case

- The Dutch **CA** DigiNotar was compromised; fraudulent certificates for google.com were issued and used to intercept users.
- Mozilla and the other vendors removed the DigiNotar roots: **every certificate of that hierarchy stopped being trusted at once.**
- The company was declared bankrupt within weeks.

The trust store is the final authority.

How a browser trusts a website (1/2)

Certificate path validation (**RFC** (Request for Comments) 5280, section 6): from the leaf up to a root already in the trust store

How a browser decides to trust `https://intranet.northmaple.internal`

TRUST STORE of the workstation (installed before any connection)

ROOT CA CERTIFICATE

NorthMaple Lab Root CA

Issuer = Subject (self-signed) · CA:TRUE · 10 years
private key kept offline (HSM)

signs

verified with the public key above

INTERMEDIATE CA CERTIFICATE (public PKIs)

Issuing CA

Issuer: Root CA · CA:TRUE
Example 1 signs the leaf directly with the root

signs

verified with the public key above

LEAF (END-ENTITY) CERTIFICATE

intranet.northmaple.internal

Issuer: Issuing CA · CA:FALSE · 200 days
SAN: DNS:intranet.northmaple.internal · EKU: serverAuth

The server sends the leaf and the intermediates.
The root is never needed on the wire: the client already holds it.

THE EIGHT CHECKS (all must pass)

- 1 Build the chain to a trusted root**
NET::ERR_CERT_AUTHORITY_INVALID
- 2 Verify every signature**
issuer public key, link by link
- 3 Check the validity dates**
NET::ERR_CERT_DATE_INVALID
- 4 Host name listed in the SAN**
NET::ERR_CERT_COMMON_NAME_INVALID
- 5 Basic Constraints and EKU**
CA:TRUE above, serverAuth on the leaf
- 6 Revocation status**
CRL, OCSP or browser-pushed lists
- 7 Browser policy**
Certificate Transparency, maximum validity
- 8 Proof of possession**
CertificateVerify signed with the server private key

CA = Certificate Authority · HSM = Hardware Security Module · SAN = Subject Alternative Name · EKU = Extended Key Usage · CRL = Certificate Revocation List
OCSP = Online Certificate Status Protocol

Cyber Defense (420-D03-AB) · Class 15 · Cyber.SoHo



Read the figure

- The server sends its **leaf** and its **intermediate** certificates.
- The browser follows each Issuer upward until it reaches a **root** that is already in its trust store.
- Every check must succeed: one failure = a full-page warning, and **no request is sent**.



How a browser trusts a website (2/2)

Eight checks — every one must succeed before the padlock appears

1 Build the chain

Follow each Issuer to a root in the trust store.

NET::ERR_CERT_AUTHORITY_INVALID

2 Verify every signature

With the public key of the next certificate up the chain.

3 Check the validity period

Now must lie between Not Before and Not After.

NET::ERR_CERT_DATE_INVALID

4 Check the name

The host name must be in the **SAN** (Subject Alternative Name) of the leaf; the **CN** (Common Name) is no longer used.

NET::ERR_CERT_COMMON_NAME_INVALID

5 Constraints and usage

Every issuer **CA:TRUE**; the leaf **EKU** (Extended Key Usage) allows server authentication.

6 Check revocation

No evidence that the issuer revoked the certificate (Lesson 3).

7 Apply browser policy

Recorded in **Certificate Transparency** logs; validity within the current maximum.

8 Proof of possession

The server signs the handshake with its private key (Lesson 2).

Self-signed certificates in the NorthMaple lab



Signed by its own key: Issuer = Subject

A **self-signed certificate** still allows encryption, but no other party confirms the identity: the browser warns. You met one in LAB_05 — the pfSense page at <https://192.168.10.1>.

Acceptable in an isolated lab if:

- 1 its **fingerprint (SHA-256)** (Secure Hash Algorithm, 256-bit output) of the whole certificate) is compared with the value on the server console before you accept it, or
- 2 the certificate — better, a lab root **CA** (Certificate Authority) certificate — is imported into the trust store of the lab machines.



Never train users to click through certificate warnings: a user used to ignoring them cannot recognize a real **MITM** (Man-in-the-Middle) attack.



Try it: broken on purpose

Ordinary browsing only — no scanning. Which check fails?

expired.badssl.com

→ validity period (check 3)

wrong.host.badssl.com

→ name (check 4)

self-signed.badssl.com

→ chain: no trusted root (check 1)

untrusted-root.badssl.com

→ chain: unknown root (check 1)

<https://badssl.com/>

Watch: Lesson 1 videos



YouTube · Professor Messer

Public Key Infrastructure — CompTIA Security+ SY0-701 1.4

≈ 8–9 min

✓ Link confirmed Oct 6, 2026

Defines **PKI** (Public Key Infrastructure) and the role of a **CA** (Certificate Authority), and why key management is the real work. Watch it before Example 1.

<https://www.youtube.com/watch?v=xHAMEF7-inQ>



YouTube · Computerphile

Public Key Cryptography

≈ 6 min

✓ Link confirmed Oct 6, 2026

A clear recap of key pairs from Class 14: the reason certificates exist at all.

https://www.youtube.com/watch?v=GSIDS_lvRv4

TikTok — search only, no direct link: #PKI #digitalcertificate #certificateauthority #chainoftrust · Before trusting a short video: Who published it? Does it agree with an official source? Is it current? Does it ask you to act outside your lab?

i Third-party videos are educational, not verified or endorsed by Cyber.SoHo; the terms of each platform apply (Legal Notice §12). Verify every link first with the VirusTotal **URL** (Uniform Resource Locator) scanner <https://www.virustotal.com/gui/home/url> (Legal Notice §10).



Example 1: the NorthMaple lab PKI (1/2)

Why does the new intranet page show "Your connection is not private"?



Scenario

The service still uses the self-signed certificate created at installation, and no NorthMaple workstation trusts it.



Task

Build a two-level private **PKI** (Public Key Infrastructure) — a lab Root **CA** (Certificate Authority) and a server certificate for **intranet.northmaple.internal** — then validate that certificate exactly as a browser does.

A

Root **CA**: **EC** (Elliptic Curve) P-256 key + self-signed certificate, **CA: TRUE**, 10 years

B

Server: key + **CSR** (Certificate Signing Request) with the **SAN** (Subject Alternative Name); the **CA** signs a 200-day certificate

C

Validate four ways: trusted ✓ · issuer unknown (error 20) · right name ✓ · wrong name (error 62)

D

One-command self-signed certificate — the model LAB_06 accepts

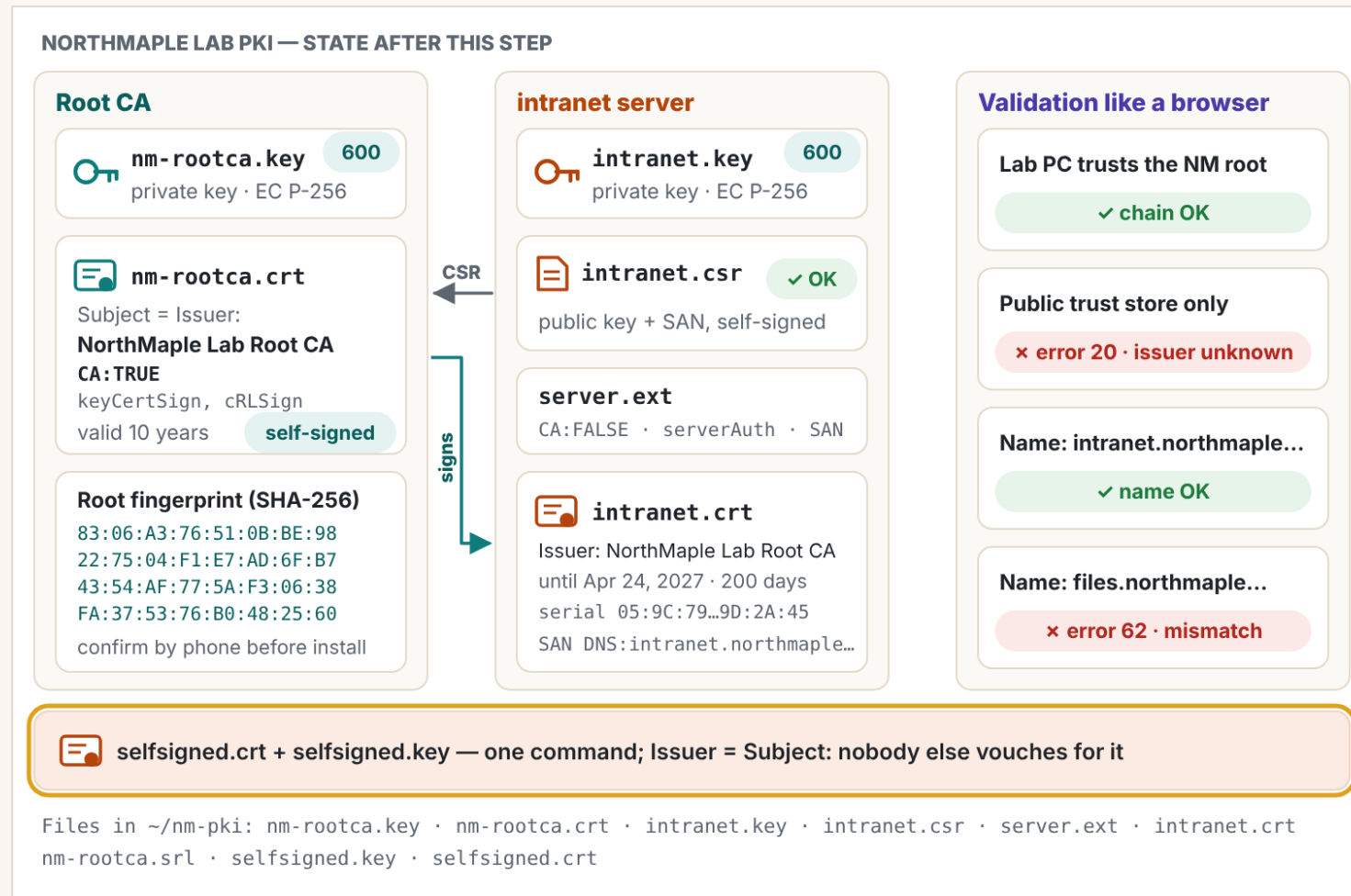
```
openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-256 ...
openssl req -x509 -new -key nm-rootca.key -days 3650 ...
openssl req -new -key intranet.key -addext "subjectAltName=DNS:..."
openssl x509 -req -in intranet.csr -CA nm-rootca.crt -days 200 ...
openssl verify -CAfile nm-rootca.crt intranet.crt
openssl x509 -in nm-rootca.crt -noout -fingerprint -sha256
```



Isolated lab only: run the scripts in your own lab **VM** (Virtual Machine) — never against a system you do not own (Legal Notice §3–§4). Its files are the inputs of LAB_06 Part 2.



Example 1: what the run proves (2/2)



Lab workstation trusts the NorthMaple root

intranet.crt: OK



Public trust store only

error 20 — unable to get local issuer certificate



Wrong name: files.northmaple.internal

error 62 — hostname mismatch



The root fingerprint is read by phone **before** installing the root on the lab workstations.

SSL and TLS: one family, several versions

Version	Published	Status today
SSL (Secure Sockets Layer) 2.0	1995	Prohibited
SSL 3.0	1996	Prohibited
TLS (Transport Layer Security) 1.0	1999	Deprecated by RFC (Request for Comments) 8996 (2021)
TLS 1.1	2006	Deprecated by RFC 8996 (2021)
TLS 1.2	2008	Allowed where needed for compatibility
TLS 1.3	2018 (RFC 8446), revised 2026 (RFC 9846)	Recommended



Canadian guidance

The Canadian Centre for Cyber Security (**ITSP.40.062** (Information Technology Security Practitioner publication 40.062)) recommends **TLS 1.3**, accepts **TLS 1.2** where compatibility needs it, and requires the phase-out of older **TLS** versions and of every **SSL** version.



"**SSL** certificate" on vendor websites = the same X.509 certificate that **TLS** uses. A certificate does not depend on the protocol version.

What HTTPS adds to HTTP

HTTP (Hypertext Transfer Protocol) carried inside a **TLS** (Transport Layer Security) connection, normally on port 443



Confidentiality

Application data is encrypted with a symmetric algorithm, for example **AES-GCM** (Advanced Encryption Standard in Galois/Counter Mode) or ChaCha20-Poly1305.



Integrity

Every **TLS** record carries an authentication tag — **TLS 1.3** uses only **AEAD** (Authenticated Encryption with Associated Data) ciphers; any change is detected and the connection is closed.



Server authentication

The server proves that it holds the private key matching a certificate the client trusts. If the server also asks for a client certificate: **mTLS** (mutual Transport Layer Security).

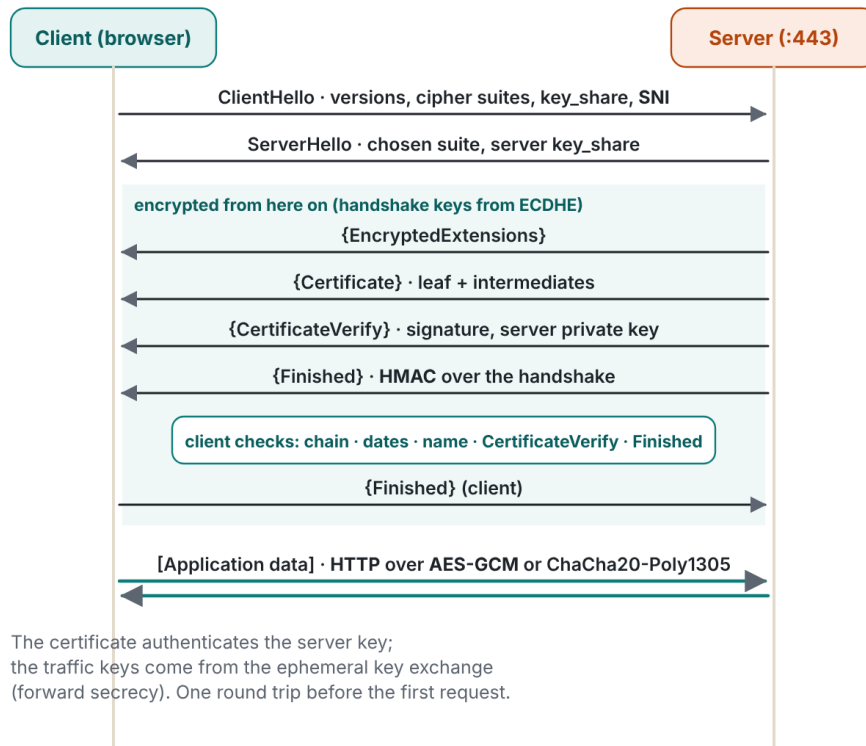


The certificate does not encrypt the web traffic. It authenticates the public key of the server. The keys that encrypt the traffic are created fresh, for every connection, by a **key exchange**.

The TLS 1.3 handshake (1/2)

Everything after ServerHello is encrypted · right side of the figure: how **SSH** (Secure Shell) trusts a host key

TLS 1.3 handshake (left) and SSH host-key trust (right)



TLS = Transport Layer Security · SSH = Secure Shell · ECDHE = Elliptic Curve Diffie-Hellman Ephemeral · SNI = Server Name Indication
 HMAC = Hash-based Message Authentication Code · HTTP = Hypertext Transfer Protocol · AES-GCM = Advanced Encryption Standard in Galois/Counter Mode · TOFU = Trust On First Use
 MITM = Man-in-the-Middle

SSH — TOFU (Trust On First Use)

FIRST CONNECTION

Server signs with its host key · client shows the SHA256 fingerprint



LATER CONNECTIONS



Alternative without **TOFU**: an **SSH** certificate authority signs host keys and user keys (ssh-keygen -s).



Three things to see

- **One round trip**: the key shares are exchanged in the first two messages.
- The server's certificate and its proof of identity travel **encrypted**.
- **SSH** reaches the same goal without a **CA** (Certificate Authority) (section 2.5).



The TLS 1.3 handshake (2/2)

1

ClientHello — versions, cipher suites, random, key share (X25519 or hybrid), server name (**SNI** (Server Name Indication))

2

ServerHello — **TLS** (Transport Layer Security) 1.3, **TLS_AES_256_GCM_SHA384**, its key share → shared secret by **ECDHE** (Elliptic Curve Diffie-Hellman Ephemeral)

3

EncryptedExtensions — remaining settings; encrypted from here on

4

Certificate — the leaf and the intermediate certificates

5

CertificateVerify — signature over the whole transcript = **proof of possession**

6

Finished — an **HMAC** (Hash-based Message Authentication Code) over the handshake: nothing was modified

7

The client validates the chain, verifies CertificateVerify, checks Finished, sends its **Finished**

8

Application data — keys derived from the shared secret by **HKDF** (HMAC-based Key Derivation Function)



Forward secrecy

Session keys come from ephemeral key shares deleted after the connection: recorded traffic stays secret even if the server's private key is stolen later. Default in **TLS 1.3**.



Hybrid post-quantum

Since Chrome 131, X25519 is combined with **ML-KEM-768** (Module-Lattice-Based Key-Encapsulation Mechanism, parameter set 768), so that recorded traffic also resists a future quantum computer (Lesson 4).

Certificates in other secure services



LDAPS (Lightweight Directory Access Protocol over TLS)

TCP (Transmission Control Protocol) port 636 — directory queries to Active Directory domain controllers.



SMTP (Simple Mail Transfer Protocol) **and IMAP** (Internet Message Access Protocol)

With **STARTTLS**: mail transfer between servers and mailbox access.



RDP (Remote Desktop Protocol)

Windows Remote Desktop presents a **TLS** (Transport Layer Security) certificate that identifies the host.



VPNs (Virtual Private Networks)

OpenVPN and **IPsec** (Internet Protocol Security) with **IKEv2** (Internet Key Exchange version 2) can use certificates. **WireGuard (LAB_06)** uses none: each peer is a static public key.



WPA3 (Wi-Fi Protected Access 3)-**Enterprise**

With **EAP-TLS** (Extensible Authentication Protocol with Transport Layer Security), the Wi-Fi network and each client authenticate with certificates (Class 13).



Management interfaces

The pfSense or OPNsense web interface is served over **HTTPS** (Hypertext Transfer Protocol Secure) (LAB_05).

SSH: a different trust model

TCP (Transmission Control Protocol) port 22 — no X.509 certificates by default: trust is built on host keys

First connection

- 1 The server signs the key-exchange data with its **host key**.
- 2 The client shows the key **fingerprint** (`SHA256:...`); compare it with the console value from `ssh-keygen -lf`.
- 3 Accepted keys are stored in `~/.ssh/known_hosts` — **TOFU** (Trust On First Use).

Later connections

A different key → **WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!**
— the server was reinstalled, or someone is intercepting. Find out which before continuing.



Users

A password or, better, a user key pair listed in `~/.ssh/authorized_keys`.



SSH (Secure Shell) certificates

An **SSH CA** (Certificate Authority) key signs host and user keys: no **TOFU** needed.



OpenSSH 10.0 (April 9, 2025)

Default key agreement: `m1kem768x25519-sha256` (hybrid post-quantum).

HTTPS with TLS compared with SSH

	HTTPS (Hypertext Transfer Protocol Secure) with TLS (Transport Layer Security)	SSH (Secure Shell)
Default port	TCP (Transmission Control Protocol) 443	TCP 22
What identifies the server	An X.509 certificate signed by a CA (Certificate Authority)	A host public key (or an OpenSSH certificate)
How the client decides to trust	The chain reaches a root in the trust store	The fingerprint is already in known_hosts (TOFU (Trust On First Use)), or an SSH CA signed the host key
Warning when something is wrong	Full-page browser warning	REMOTE HOST IDENTIFICATION HAS CHANGED!
Typical user authentication	A login form inside the encrypted session, or a client certificate	A password or a user key pair
Session keys	Ephemeral ECDHE (Elliptic Curve Diffie-Hellman Ephemeral); hybrid with ML-KEM (Module-Lattice-Based Key-Encapsulation Mechanism) in current browsers	Ephemeral key exchange; hybrid with ML-KEM by default since OpenSSH 10.0

Watch: Lesson 2 videos



YouTube · ByteByteGo

SSL, TLS, HTTPS Explained

≈ 6 min

✓ Link confirmed Oct 6, 2026

Animated walk-through of the handshake: certificate check, key exchange, then symmetric encryption.

<https://www.youtube.com/watch?v=j9QmMEWmcfo>



YouTube · Computerphile

How Secure Shell Works (SSH)

≈ 9 min

✓ Link confirmed Oct 6, 2026

How **SSH** (Secure Shell) agrees on keys and authenticates the server and the user — the model behind LAB_03 **SSH** hardening.

<https://www.youtube.com/watch?v=ORcvSkgdA58>

TikTok — search only, no direct link: #TLS #HTTPS #SSH #TLShandshake · Before trusting a short video: Who published it? Does it agree with an official source? Is it current? Does it ask you to act outside your lab?

i Third-party videos are educational, not verified or endorsed by Cyber.SoHo; the terms of each platform apply (Legal Notice §12). Verify every link first with the VirusTotal **URL** (Uniform Resource Locator) scanner <https://www.virustotal.com/gui/home/url> (Legal Notice §10).

Example 2: the first milliseconds of HTTPS (1/2)

What happens when an accountant opens `https://intranet.northmaple.internal`?



Scenario

Prove four facts to the owner of the firm with one script — on `127.0.0.1` only: no packet leaves the machine.



Task

Start an **HTTPS** (Hypertext Transfer Protocol Secure) test service with the Example 1 certificate, then connect with four different clients.



1 · Trusts the NorthMaple root

TLSv1.3 · `TLS_AES_256_GCM_SHA384`
· page delivered



2 · Public trust store only

REFUSED — unable to get local issuer certificate



3 · Trusted root, wrong name

REFUSED — Hostname mismatch for `files.northmaple.internal`



4 · Plain HTTP (Hypertext Transfer Protocol) to the HTTPS port

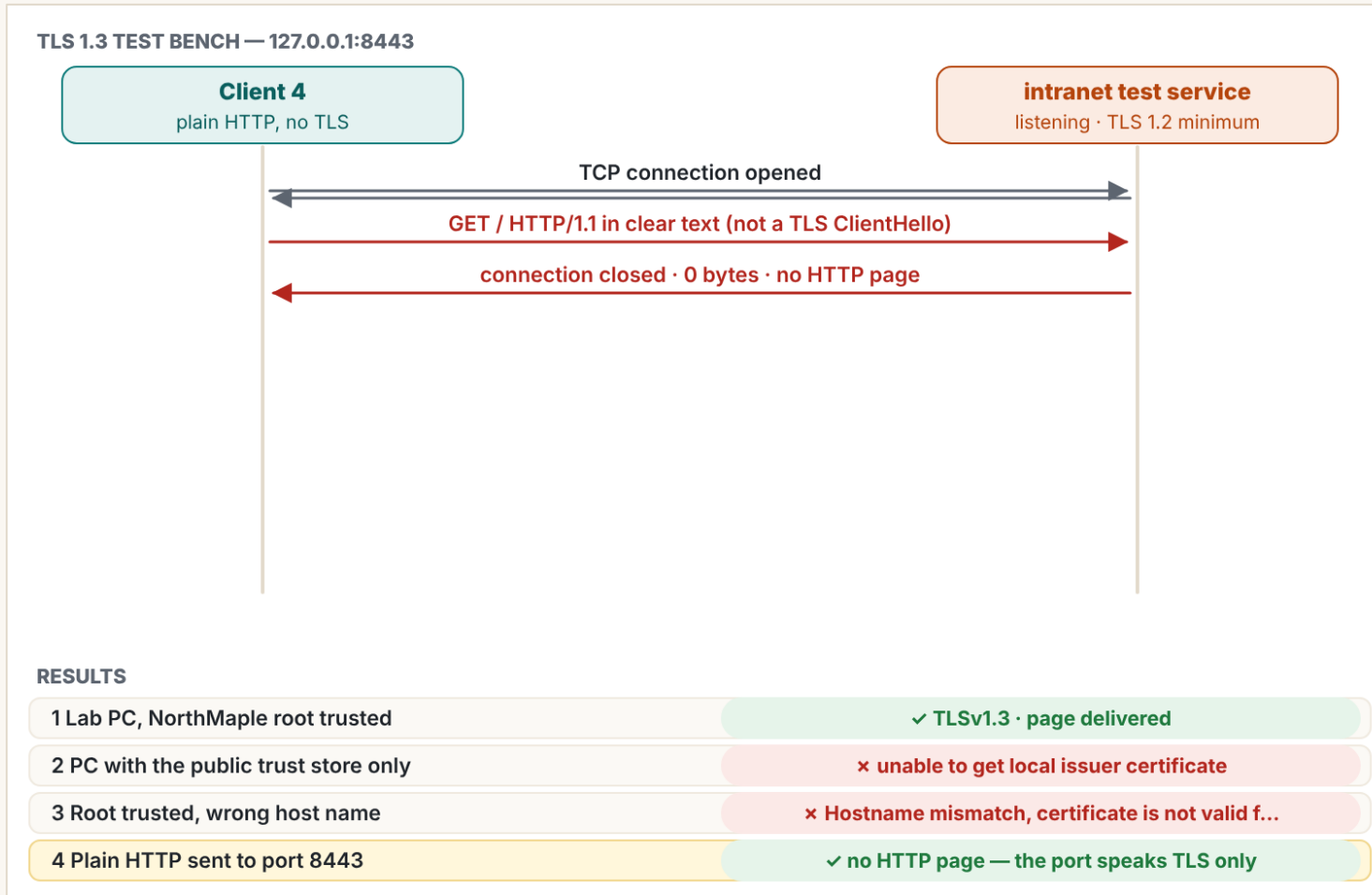
No **HTTP** page — the **TLS** (Transport Layer Security) service closed the connection (0 bytes)



Isolated lab only: run the scripts in your own lab **VM** (Virtual Machine) — never against a system you do not own (Legal Notice §3–§4). LAB_06 Part 2 pattern: **HTTPS** only.



Example 2: what the run proves (2/2)



State after test 4

One client trusted, three refused — each for a different reason.



Evidence for LAB_06

Paste the console output in your validation notes (Manual Chapter 6, section N.4).



Remote worker on public Wi-Fi: **HTTPS** (Hypertext Transfer Protocol Secure) inside the **VPN** (Virtual Private Network) = **two independent layers** of encryption (resilience row 6).

The certificate life cycle: nine stages

Keys get exposed, identities change, algorithms age — and an unrenewed certificate stops a service

1 Key generation



On the system that will use it, or in an **HSM** (Hardware Security Module); the private key is never sent to the **CA** (Certificate Authority).

2 Enrollment



A **CSR** (Certificate Signing Request): public key + names, signed with the private key = proof of possession.

3 Validation



The **CA** or its **RA** (Registration Authority) checks control of the names (**ACME** (Automatic Certificate Management Environment) [http-01](#) or [dns-01](#)).

4 Issuance



The **CA** signs; public **CAs** (Certificate Authorities) also record the certificate in Certificate Transparency logs.

5 Installation



Certificate + chain on the server; the private key file is owner-only (LAB_02).

6 Monitoring



Inventory: where, owner, key algorithm, expiry date; alert well before expiry.

7 Renewal



A new certificate before Not After — with a new key pair: **re-keying**.

8 Revocation



The **CA** declares the certificate invalid before its expiry date.

9 Expiry and retirement

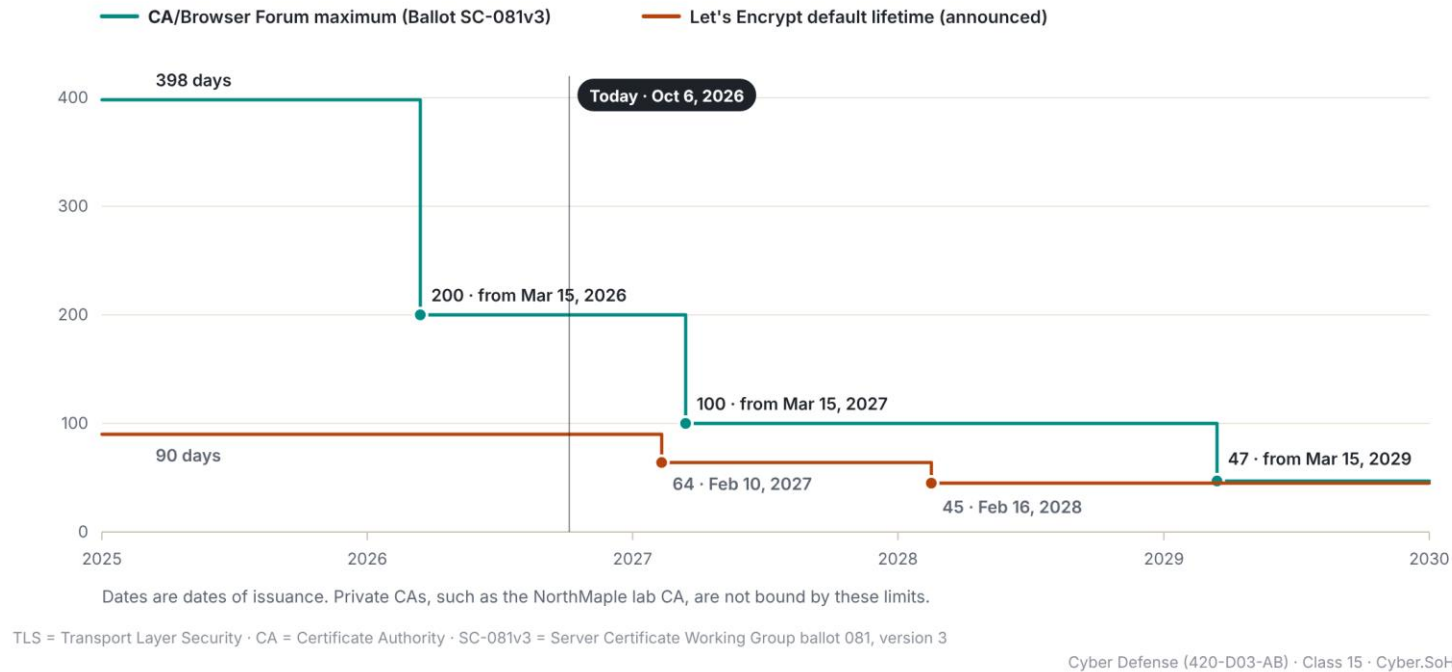


The certificate stops being valid; the old private key is deleted securely.

How long a certificate may live

The maximum allowed by browsers and **CAs** (Certificate Authorities), and the Let's Encrypt default lifetimes

Maximum validity of public TLS certificates, in days



Ballot SC-081v3 (Server Certificate Working Group ballot 081, version 3), **April 11, 2025**

- Issued from March 15, 2026: **200 days** — in force today
- From March 15, 2027: **100 days**
- From March 15, 2029: **47 days**
- Let's Encrypt: **90 days** today → **64** (Feb 10, 2027) → **45** (Feb 16, 2028)



Manual renewal does not scale: automate with an **ACME** (Automatic Certificate Management Environment) client (certbot, acme.sh, the **ACME** package of pfSense). **A short validity limits how long a stolen key can be misused.**

Revocation (1/2): why a certificate is withdrawn

Reason codes of **RFC** (Request for Comments) 5280, section 5.3.1 — Example 3 uses **keyCompromise**

Code	Reason	When it is used
0	unspecified	No reason given.
1	keyCompromise	The private key of the subject was exposed or stolen.
2	cACompromise	The private key of the CA (Certificate Authority) itself was compromised.
3	affiliationChanged	The subject name or organization changed.
4	superseded	The certificate was replaced by a new one.
5	cessationOfOperation	The service or host was shut down.
6	certificateHold	Temporary suspension; not allowed for public TLS (Transport Layer Security) certificates.
8	removeFromCRL	Used only in delta CRLs (Certificate Revocation Lists) to cancel a hold.
9	privilegeWithdrawn	A privilege granted by the certificate was withdrawn.
10	aACompromise	An attribute authority was compromised.

Revocation (2/2): how clients learn about it



CRL (Certificate Revocation List)

A signed list of revoked serial numbers, dates and reasons, published at the **CRL** Distribution Points address.

+ simple, cacheable, private – size, delay until Next Update



OCSP (Online Certificate Status Protocol)

The client asks the **CA** (Certificate Authority)'s responder about one serial: good, revoked or unknown (**RFC** (Request for Comments) 6960).

+ current – the **CA** sees which sites you visit; "soft fail"

OCSP stapling: the server attaches the signed answer to the handshake.



Browser-distributed data

Browsers download compressed revocation information in the background.

No query is sent while the user browses.

2023

Ballot **SC-063** (Server Certificate Working Group ballot 063): **OCSP** optional, **CRLs** (Certificate Revocation Lists) mandatory for public **CAs** (Certificate Authorities)

May 7, 2025

Let's Encrypt removes **OCSP** addresses from its certificates

Aug 6, 2025

Let's Encrypt shuts down its **OCSP** service

Today

Shorter lifetimes shrink the window in which a revoked certificate is still accepted



The life cycle in the NorthMaple laboratory



pfSense Certificate Manager

Can act as the internal **CA** (Certificate Authority) of NorthMaple: it creates **CAs** (Certificate Authorities), server and user certificates, and **CRLs** (Certificate Revocation Lists).



WireGuard: no CRL (Certificate Revocation List)

Each peer is a public key in the firewall configuration. Lost laptop = **delete that peer's public key on NM-FW01** + a new key pair for the replacement.



Write it in your manual

The peer-removal procedure answers row 5 of the resilience assessment: "A company laptop with client data is lost or stolen".

The certificate register — keep it next to the LAB_05 device registry

Certificate	Host	Issuer	Serial	Not After	Planned renewal	Owner
intranet.northmaple.internal	internal web host	NorthMaple Lab Issuing CA	1001	Apr 24, 2027	Feb 16, 2027	IT (Information Technology) administrator

Watch: Lesson 3 video



YouTube · Professor Messer

Certificates — CompTIA Security+ SY0-701 1.4

≈ 14 min

✓ Link confirmed Oct 6, 2026

CSR (Certificate Signing Request) generation, self-signed and wildcard certificates, **CRL** (Certificate Revocation List), **OCSP** (Online Certificate Status Protocol) and **OCSP** stapling, in one sequence.

https://www.youtube.com/watch?v=cLa94BZH_9s



Watch it after class, then answer the Lesson 3 practice questions in the lecture page.



TikTok — search only, no direct link: #SSLCertificate #certificaterevocation #OCSP #LetsEncrypt · Before trusting a short video: Who published it? Does it agree with an official source? Is it current? Does it ask you to act outside your lab?

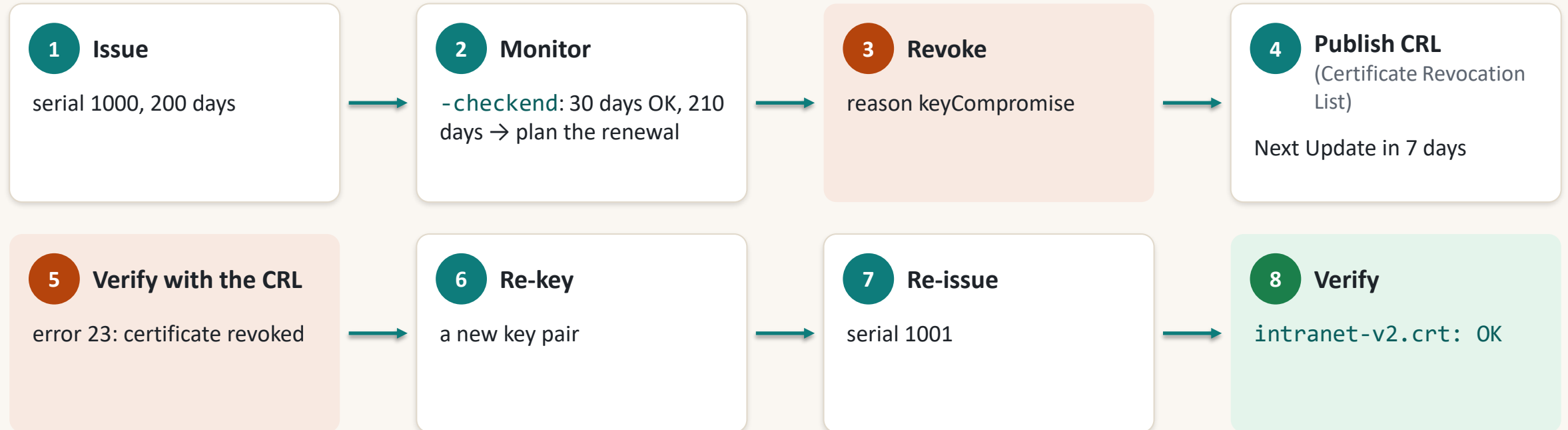


Third-party videos are educational, not verified or endorsed by Cyber.SoHo; the terms of each platform apply (Legal Notice §12). Verify every link first with the VirusTotal **URL** (Uniform Resource Locator) scanner <https://www.virustotal.com/gui/home/url> (Legal Notice §10).



Example 3: a stolen laptop held the key (1/2)

Friday, 17:40 — the technician's laptop held a copy of intranet.key



WireGuard VPN (Virtual Private Network): the administrator also deletes the public key of the stolen laptop from NM-FW01 (resilience assessment, rows 5 and 9).



Isolated lab only: run the scripts in your own lab **VM** (Virtual Machine) — never against a system you do not own (Legal Notice §3–§4). Delete ~/nm-ca before a second run.



Example 3: what the run proves (2/2)

LIFE CYCLE OF intranet.northmaple.internal

✓ CA ready ✓ Key + CSR ✓ Issued ✓ Monitor ✓ Revoked ✓ CRL ✓ Re-key ✓ Re-issued

index.txt — CA database serial next: 1002

R serial 1000 · expires 2027-04-24
revoked 2026-10-06 · keyCompromise

V serial 1001 · expires 2027-04-24
intranet-v2 (new key pair)

Expiry monitoring (v1)

+30 d: valid

Oct 6, 2026 Apr 24, 2027

+210 d: past Not After → renew
revoked on day 0 (keyCompromise)

nm-ca.crl — Certificate Revocation List

CRL number 1000 · signed by NorthMaple Lab Issuing CA
Last Update Oct 6, 2026 · Next Update Oct 13, 2026

serial 1000 · revoked Oct 6, 2026 · Key Compromise

Client check with the CRL (-crl_check)

intranet-v1.crt ✗ error 23 · certificate revoked

intranet-v2.crt ✓ OK · not on the CRL

FILES IN ~/nm-ca

index.txt serial crlnumber ca.cnf ca.key ca.crt intranet-v1.key intranet-v1.csr

intranet-v1.crt newcerts/1000.pem nm-ca.crl ca-with-crl.pem intranet-v2.key intranet-v2.csr

intranet-v2.crt newcerts/1001.pem



index.txt: V → R

Serial 1000 revoked — keyCompromise



Old certificate

error 23 — certificate revoked



New certificate

intranet-v2.crt: OK — serial 1001



The model for the renewal and revocation notes of your LAB_06 certificate.



No single best algorithm, only a best fit



VPN (Virtual Private Network)

needs a fast bulk cipher and a key exchange



Root CA (Certificate Authority)

needs a signature key trustworthy for 10 years or more



Password storage

needs a deliberately slow function



File checksum

needs collision resistance — nothing else

The eight assessment criteria

1



Security strength

2



Protection lifetime

3



Official guidance

4



Performance and size

5



Correct use

6



Implementation assurance

7



Key management cost

8

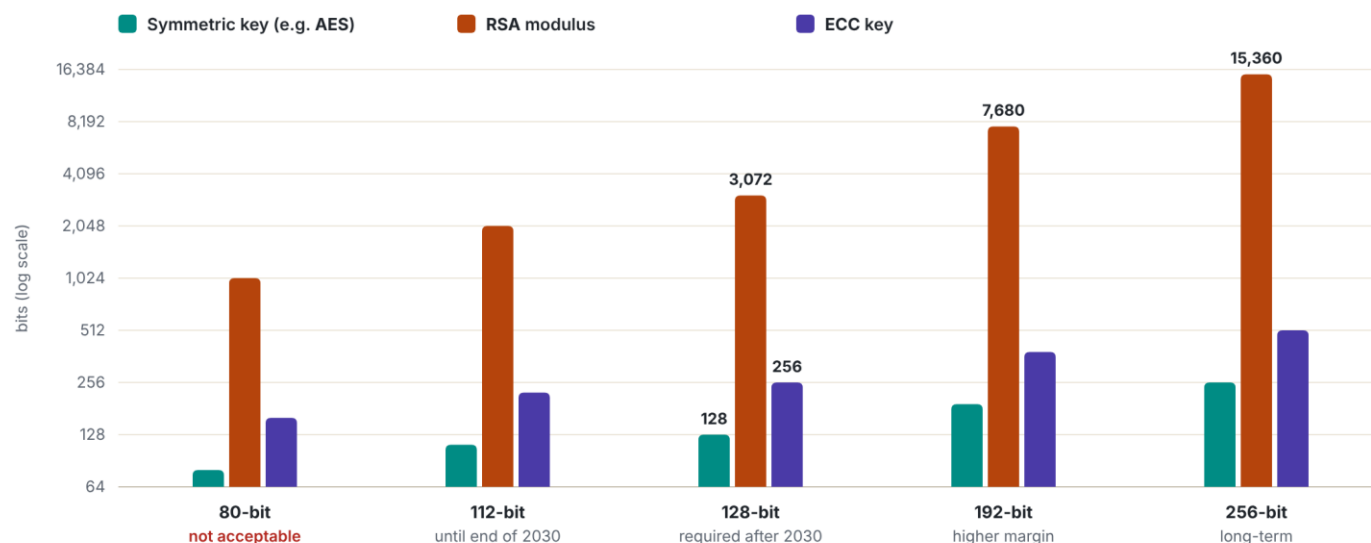


Quantum resistance

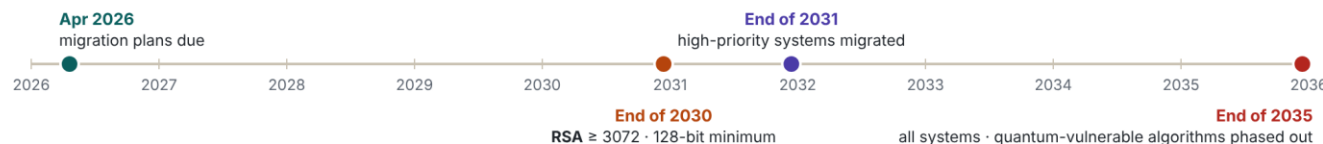
Criterion 1: security strength

n bits means about 2^n operations for the best known attack — each extra bit doubles the work

Key size needed for the same security strength (NIST SP 800-57 Part 1 Rev. 5)



Canadian milestones (Cyber Centre ITSP.40.111 v5 and post-quantum roadmap)



NIST = National Institute of Standards and Technology · SP = Special Publication · AES = Advanced Encryption Standard · RSA = Rivest-Shamir-Adleman
ECC = Elliptic Curve Cryptography · ITSP = Information Technology Security Practitioner series

Cyber Defense (420-D03-AB) · Class 15 · Cyber.SoHo



128 bits of strength

- = **AES** (Advanced Encryption Standard) with a 128-bit key (symmetric)
- = **ECC** (Elliptic Curve Cryptography) with a 256-bit key: P-256, Curve25519
- = **RSA** (Rivest-Shamir-Adleman) with a 3072-bit modulus
- Elliptic curves reach 128 bits with much shorter keys: **TLS** (Transport Layer Security) 1.3, **SSH** (Secure Shell) and WireGuard prefer them.
- 80 bits (**RSA-1024** (Rivest-Shamir-Adleman with a 1024-bit modulus), **SHA-1** (Secure Hash Algorithm 1) collisions): **not acceptable**.

Criteria 2 and 3: lifetime and guidance



2 · Protection lifetime

- How long the data must stay confidential, or a signature verifiable.
- NorthMaple keeps client tax records **at least six years** (CRA (Canada Revenue Agency) rule): protection chosen in 2026 must still hold in **2032**.
- **Harvest now, decrypt later:** traffic recorded today is decrypted once the key exchange can be broken — the main reason for the post-quantum migration.



3 · Official guidance: ITSP.40.111 v5



Approved: **AES** (Advanced Encryption Standard) with 128, 192 or 256-bit keys; **SHA-2** (Secure Hash Algorithm 2 family) and **SHA-3** (Secure Hash Algorithm 3 family); the post-quantum **ML-KEM** (Module-Lattice-Based Key-Encapsulation Mechanism), **ML-DSA** (Module-Lattice-Based Digital Signature Algorithm) and **SLH-DSA** (Stateless Hash-Based Digital Signature Algorithm).



By the end of 2030: **RSA** (Rivest-Shamir-Adleman) keys of 3072 bits (2048 today); minimum strength 128 bits (112 today).



By the end of 2035: **RSA**, **ECDSA** (Elliptic Curve Digital Signature Algorithm) and **EdDSA** (Edwards-curve Digital Signature Algorithm) without a post-quantum replacement are phased out.



Not acceptable: **SHA-1** (Secure Hash Algorithm 1) for signatures; **MD5** (Message-Digest Algorithm 5), **DES** (Data Encryption Standard), **3DES** (Triple Data Encryption Standard), **RC4** (Rivest Cipher 4); **SSL** (Secure Sockets Layer) 3.0, **TLS** (Transport Layer Security) 1.0 and 1.1.

Criteria 4 to 7: speed, use, code, keys



4 · Performance and size

Time per operation, key and signature size, bytes added to each handshake, **CPU** (Central Processing Unit) and battery cost — measured on the target hardware (Example 4).



5 · Correct use

AES (Advanced Encryption Standard) in **ECB** (Electronic Codebook) mode reveals patterns; a reused **AES-GCM** (Advanced Encryption Standard in Galois/Counter Mode) nonce destroys confidentiality and integrity; one unsalted **SHA-256** (Secure Hash Algorithm, 256-bit output) pass for passwords is guessed billions of times per second on a **GPU** (Graphics Processing Unit).



6 · Implementation assurance

Maintained, reviewed libraries (OpenSSL, the **OS** (Operating System) cryptography), never home-made code; **FIPS** (Federal Information Processing Standard) 140-3 modules validated by the **CMVP** (Cryptographic Module Validation Program). **CIS** (Center for Internet Security) Control 3: encrypt sensitive data in transit and at rest.



7 · Key management cost

How keys are generated, stored (**TPM** (Trusted Platform Module), **HSM** (Hardware Security Module)), rotated and revoked. Keys that cannot be managed reliably make any algorithm ineffective.

Criterion 8: resistance to quantum attacks



Shor's algorithm

Would break **RSA** (Rivest-Shamir-Adleman), **DH** (Diffie-Hellman) and **ECC** (Elliptic Curve Cryptography) — whatever the key size.



Grover's algorithm

Only reduces the effective strength of symmetric keys: **AES-256** (Advanced Encryption Standard with a 256-bit key) remains adequate.



NIST (National Institute of Standards and Technology), August 13, 2024

FIPS (Federal Information Processing Standard) 203 — **ML-KEM** (Module-Lattice-Based Key-Encapsulation Mechanism)

FIPS 204 — **ML-DSA** (Module-Lattice-Based Digital Signature Algorithm)

FIPS 205 — **SLH-DSA** (Stateless Hash-Based Digital Signature Algorithm)

Mar 2025

HQC (Hamming Quasi-Cyclic) selected as a backup **KEM** (Key-Encapsulation Mechanism)

Apr 9, 2025

OpenSSH 10.0: hybrid key agreement by default

Jun 23, 2025

Government of Canada post-quantum roadmap

Apr 2026

Departmental migration plans due

End of 2031

High-priority systems migrated

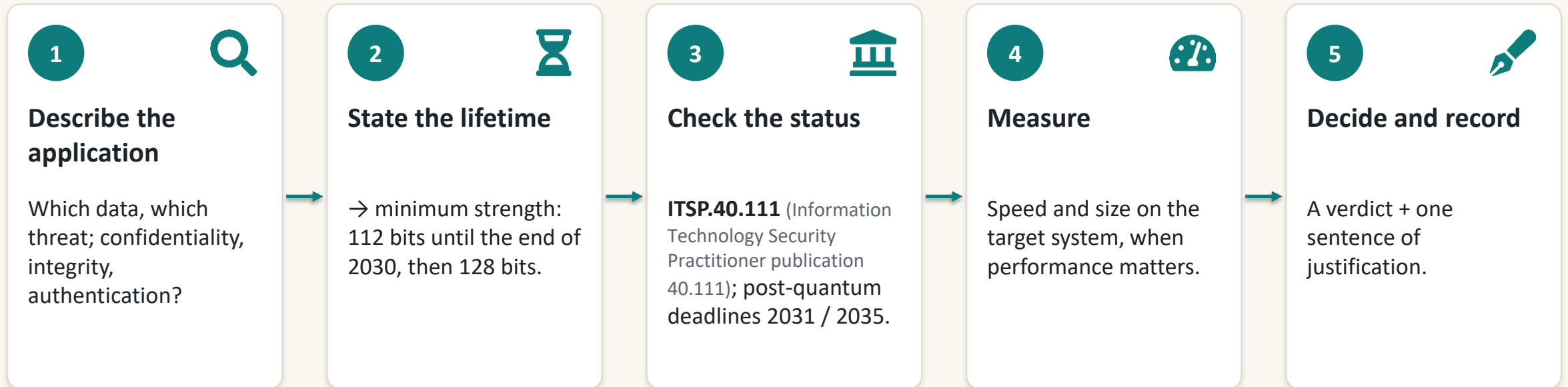
End of 2035

All systems migrated



Already deployed: hybrid X25519 + **ML-KEM-768** (Module-Lattice-Based Key-Encapsulation Mechanism, parameter set 768) in Chrome (since version 131) and in OpenSSH 10.0; WireGuard offers an optional pre-shared key that adds post-quantum resistance.

A repeatable five-step assessment



Verdicts:

Adequate

Upgrade

Plan post-quantum migration

Replace



Step 5 is the **"it is justified"** sentence that every chapter of your Final Project requires.

Same algorithm, different verdicts

Use	Verdict	Reason
SHA-256 (Secure Hash Algorithm, 256-bit output) checksum of a lab evidence file (Class 14)	Adequate	128-bit collision resistance is enough to detect a modified file.
SHA-256 , one pass, no salt, for stored passwords	Replace	Too fast: use a slow, salted password-hashing function — Argon2id, scrypt, bcrypt or PBKDF2 (Password-Based Key Derivation Function 2).
RSA-2048 (Rivest-Shamir-Adleman with a 2048-bit modulus) for a 200-day web certificate issued in 2026	Adequate	112-bit strength is accepted until the end of 2030; the certificate expires in 2027.
RSA-2048 for a root CA (Certificate Authority) valid ten years from 2026	Upgrade	Must stay trustworthy after 2030: P-384 or RSA-3072 (Rivest-Shamir-Adleman with a 3072-bit modulus) at least; plan a post-quantum or hybrid root before 2035.



Your turn: **RSA-2048** on a certificate that must stay valid until **2032** — which verdict, and why?

Watch: Lesson 4 videos



YouTube · 3Blue1Brown

How secure is 256 bit security?

≈ 5 min

✓ Link confirmed Oct 6, 2026

Makes the number 2^{256} concrete: why security strength in bits is the right measuring unit.

https://www.youtube.com/watch?v=S9JGmA5_unY



YouTube · Computerphile

Elliptic Curves

≈ 9 min

✓ Link confirmed Oct 6, 2026

Why a 256-bit elliptic-curve key matches a 3072-bit **RSA** (Rivest-Shamir-Adleman) key.

<https://www.youtube.com/watch?v=NF1pwjL9-DE>

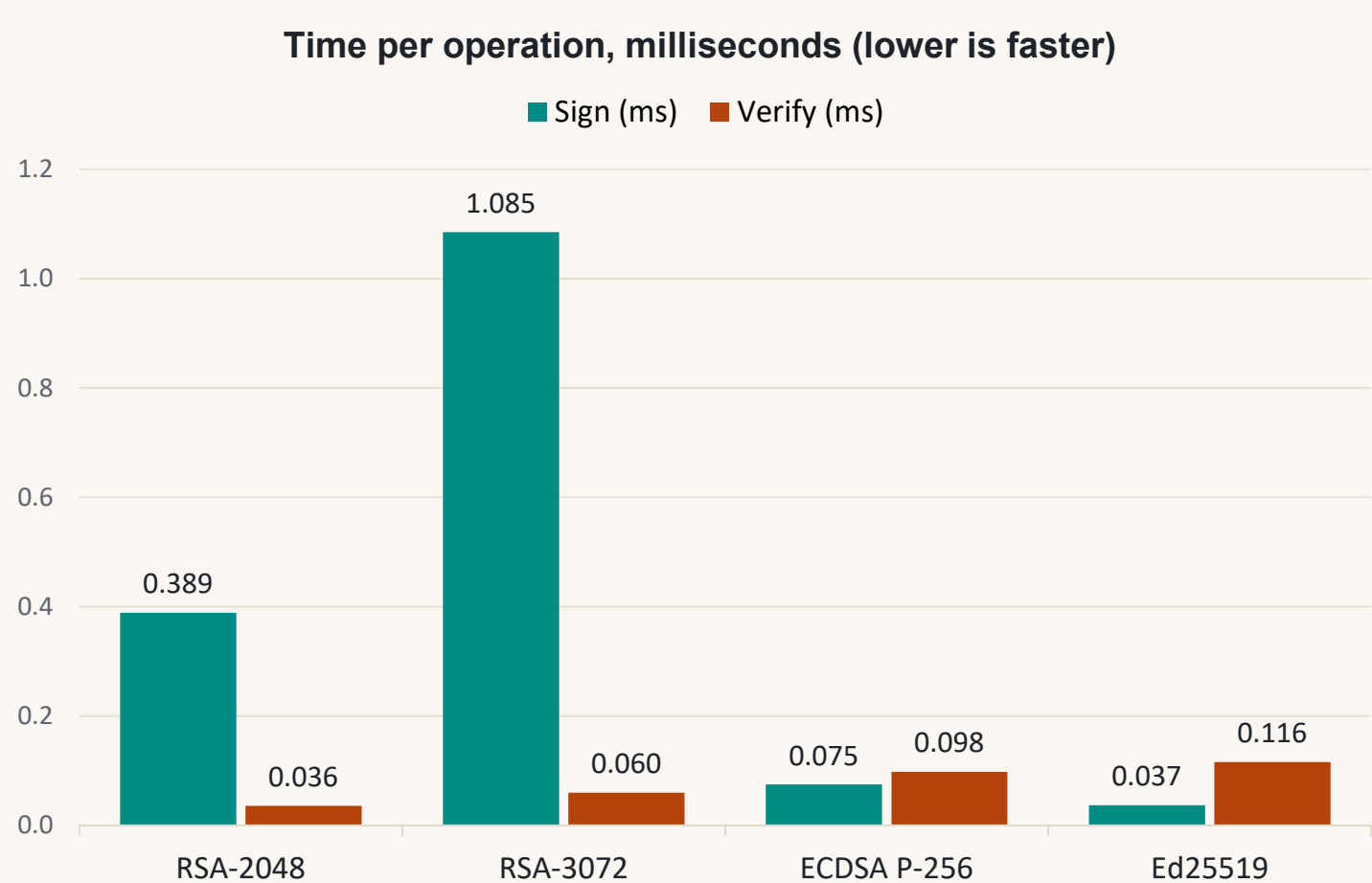
TikTok — search only, no direct link: #postquantumcryptography #encryption #RSA #ellipticcurve · Before trusting a short video: Who published it? Does it agree with an official source? Is it current? Does it ask you to act outside your lab?

i Third-party videos are educational, not verified or endorsed by Cyber.SoHo; the terms of each platform apply (Legal Notice §12). Verify every link first with the VirusTotal **URL** (Uniform Resource Locator) scanner <https://www.virustotal.com/gui/home/url> (Legal Notice §10).



Example 4: measuring signature systems (1/2)

Recorded on October 6, 2026 · 200 operations each · timings depend on the processor: compare the systems with each other



System	Bits	Signature (B)	Public key (B)
RSA-2048	112	256	294
RSA-3072	128	384	422
ECDSA P-256	128	71	91
Ed25519	128	64	44



Elliptic curves signed about **5–11× faster** than **RSA-2048** (Rivest-Shamir-Adleman with a 2048-bit modulus) here, with much smaller signatures.

RSA (Rivest-Shamir-Adleman) verifies fastest — good for clients, not for a busy server that signs every handshake.



Example 4: the NorthMaple verdict matrix (2/2)

PART 1 — MEASURED ON THIS COMPUTER (200 operations each)

all four systems measured

Signing (ms per signature)

RSA-2048	 0.389 ms
RSA-3072	 1.085 ms
ECDSA P-256	 0.075 ms
Ed25519	 0.037 ms

Verification (ms)

RSA-2048	 0.036 ms
RSA-3072	 0.060 ms
ECDSA P-256	 0.098 ms
Ed25519	 0.116 ms

Signature size (bytes)

RSA-2048	 256 B
RSA-3072	 384 B
ECDSA P-256	 71 B
Ed25519	 64 B

PART 2 — VERDICT MATRIX FOR NORTHMAPLE

Application	System	Until	Verdict
Lab Root CA signature (10-year cert)	ECDSA P-256	2036	◆ Plan post-quantum
Intranet HTTPS certificate (200 days)	ECDSA P-256	2027	✓ Adequate
Old file-server certificate	RSA-2048	2033	▲ Upgrade
WireGuard VPN key exchange	X25519	2032	◆ Plan post-quantum
WireGuard VPN data encryption	ChaCha20-Poly1305	2032	✓ Adequate
Encrypted client-records folder	AES-256	2032	✓ Adequate
SSH host key on NM-LX01	Ed25519	2030	✓ Adequate
Passwords in an old internal app	SHA-256	2032	✗ Replace
Checksums of lab evidence files	SHA-256	2027	✓ Adequate

5

Adequate

1

Upgrade

2

Plan post-quantum migration

1

Replace

- **Upgrade:** the old **RSA-2048** (Rivest-Shamir-Adleman with a 2048-bit modulus) certificate is used after 2030.
- **Plan post-quantum:** the WireGuard key exchange (records kept to 2032) and the 10-year root (to 2036).
- **Replace:** unsalted **SHA-256** (Secure Hash Algorithm, 256-bit output) password hashes → Argon2id, scrypt, bcrypt or **PBKDF2** (Password-Based Key Derivation Function 2).





LAB_06 preview: VPN and encryption

Summary only — the lab kit is distributed separately · 5% of the course grade · the last hands-on lab



Part 1 — Class 16 · Wednesday, October 7

- Remote-access **VPN** (Virtual Private Network) into the company network — **WireGuard** recommended; OpenVPN or the pfSense / OPNsense **VPN** accepted.
- Keys, users (peers) and allowed networks; connect NM-RW01 and verify.



Part 2 — Class 17 · Saturday, October 10

- Encrypt one folder or volume of company data with a free tool.
- Enable **HTTPS** (Hypertext Transfer Protocol Secure) with a **self-signed certificate** on one internal service.
- Validate: **VPN** traffic encrypted · **HTTPS** only · folder unreadable without its key — record the evidence.



Deliverable: Chapter 6 — Secure Communication (Final Project + Course Procedure Manual), sections N.1 to N.6; N.6 = the **AI** (Artificial Intelligence) disclosure table or "No **AI** tools were used in this chapter."

Example 1

→ certificate files + fingerprint check (Part 2)

Example 2

→ "answers only over **HTTPS**" validation (N.4)

Example 3

→ renewal and revocation notes; WireGuard peer removal

Example 4

→ the justification sentences of Chapter 6





Today's work time: prepare LAB_06



Start **NM-FW01** (pfSense) and open its dashboard over **HTTPS** (Hypertext Transfer Protocol Secure) from NM-WS01.



Confirm the addresses: **WAN** (Wide Area Network) 198.51.100.10; 192.168.10.1, 20.1, 30.1 and 40.1 on the internal zones.



Repeat two allowed/blocked tests from LAB_05: the zoning rules still apply.



Write the firewall software version in the device registry.



Take a snapshot of every lab **VM** (Virtual Machine) named **pre-LAB06**.



Bring Chapters 1 to 5 of your Course Procedure Manual up to date.



Q_04 — Class 18

- Tuesday, October 13 · 2.5% · the last quiz.
- Classes 15 to 17, conceptual level: **PKI** (Public Key Infrastructure) and certificates, what a **VPN** (Virtual Private Network) provides, encryption at rest versus in transit, how encryption is validated.



Isolated lab only — snapshots keep a mistake inside your laboratory (Legal Notice §3).





Key takeaways and next class

1

Certificates & trust

A certificate binds a public key to a name; a browser trusts it only if the chain reaches a root in its trust store and every check passes.

2

Secure protocols

TLS (Transport Layer Security) authenticates the server with its certificate and CertificateVerify; the traffic keys come from an ephemeral key exchange. **SSH** (Secure Shell) trusts host keys by fingerprint.

3

Life cycle

Certificates are issued, monitored, renewed with new keys and revoked; public lifetimes shrink to 200, 100, then 47 days — automate.

4

Assessing crypto

Judge cryptography per application: strength, lifetime, official guidance, performance, correct use — and plan the post-quantum migration.



Next: Class 16 — Wednesday, October 7, 6:00 pm EDT (Eastern Daylight Time) — **LAB_06 Part 1: remote-access VPN** (Virtual Private Network)

Before then: watch the videos, flip the flashcards, answer the practice questions — and sign the Acknowledgment and Agreement page at the start of the session.



All links (1/4)

 Provided for convenience; a link is not an endorsement. Before opening any link, verify it with the VirusTotal **URL** (Uniform Resource Locator) scanner (Legal Notice §10).

#	Organization or document	Official address (click to open)
1	IETF — the Internet standards body	https://www.ietf.org/
2	ITU-T — publisher of the X.509 recommendation	https://www.itu.int/en/ITU-T/Pages/default.aspx
3	RFC 5280 — X.509 certificate and CRL profile	https://www.rfc-editor.org/rfc/rfc5280
4	RFC 9846 — TLS 1.3 (2026)	https://www.rfc-editor.org/rfc/rfc9846
5	RFC 8446 — TLS 1.3 (2018, now obsolete)	https://www.rfc-editor.org/rfc/rfc8446
6	RFC 8996 — deprecating TLS 1.0 and TLS 1.1	https://www.rfc-editor.org/rfc/rfc8996
7	RFC 6960 — OCSP	https://www.rfc-editor.org/rfc/rfc6960
8	RFC 8555 — ACME	https://www.rfc-editor.org/rfc/rfc8555
9	CA/Browser Forum	https://cabforum.org/
10	CA/Browser Forum — Ballot SC-081v3 (April 11, 2025)	https://cabforum.org/2025/04/11/ballot-sc081v3-introduce-schedule-of-reducing-validity-and-data-reuse-periods/
11	CA/Browser Forum — Ballot SC-063v4 (2023)	https://cabforum.org/2023/07/14/ballot-sc063v4-make-ocsp-optional-require-crls-and-incentivize-automation
12	Let's Encrypt	https://letsencrypt.org/
13	ISRG — operator of Let's Encrypt	https://www.abetterinternet.org/
14	Let's Encrypt — How It Works	https://letsencrypt.org/how-it-works/
15	Let's Encrypt — Ending OCSP Support in 2025	https://letsencrypt.org/2024/12/05/ending-ocsp/
16	Let's Encrypt — lifetimes down to 45 days	https://letsencrypt.org/2025/12/02/from-90-to-45/
17	Mozilla CA Certificate Program	https://wiki.mozilla.org/CA

Abbreviations: **IETF** = Internet Engineering Task Force · **ITU-T** = International Telecommunication Union, Telecommunication Standardization Sector · **RFC** = Request for Comments · **CRL** = Certificate Revocation List · **TLS** = Transport Layer Security · **OCSP** = Online Certificate Status Protocol · **ACME** = Automatic Certificate Management Environment · **CA/Browser Forum** = Certification Authority/Browser Forum · **SC-081v3** = Server Certificate Working Group ballot 081, version 3 · **SC-063v4** = Server Certificate Working Group ballot 063, version 4 · **ISRG** = Internet Security Research Group · **CA** = Certificate Authority

All links (2/4)

 Provided for convenience; a link is not an endorsement. Before opening any link, verify it with the VirusTotal **URL** (Uniform Resource Locator) scanner (Legal Notice §10).

#	Organization or document	Official address (click to open)
18	Microsoft Trusted Root Program — requirements	https://learn.microsoft.com/en-us/security/trusted-root/program-requirements
19	CCADB	https://www.ccadb.org/
20	Mozilla Security Blog — the DigiNotar case (2011)	https://blog.mozilla.org/security/2011/08/29/fraudulent-google-com-certificate/
21	Certificate Transparency	https://certificate.transparency.dev/
22	badssl.com — test pages with broken certificates	https://badssl.com/
23	ICANN — resolution of July 29, 2024 (.internal)	https://www.icann.org/en/board-activities-and-meetings/materials/approved-resolutions-special-meeting-of-the-icann-board-29-07-2024-en
24	Google Security Blog — post-quantum key exchange	https://security.googleblog.com/2024/09/a-new-path-for-kyber-on-web.html
25	Canadian Centre for Cyber Security	https://www.cyber.gc.ca/en
26	Cyber Centre — ITSP.40.062 , network protocols	https://cyber.gc.ca/en/guidance/guidance-securely-configuring-network-protocols-itsp40062
27	Cyber Centre — ITSP.40.111 , algorithms	https://cyber.gc.ca/en/guidance/cryptographic-algorithms-unclassified-protected-protected-b-information-itsp40111
28	Cyber Centre — ITSM.40.001 , post-quantum roadmap	https://cyber.gc.ca/en/guidance/roadmap-migration-post-quantum-cryptography-government-canada-itsm40001
29	Canada Revenue Agency — keeping records	https://www.canada.ca/en/revenue-agency/services/tax/businesses/topics/keeping-records/where-keep-your-records-long-request-permission-destroy-them-early.html
30	NIST SP 800-57 Part 1 Rev. 5 — key management	https://csrc.nist.gov/pubs/sp/800/57/pt1/r5/final
31	NIST SP 800-52 Rev. 2 — TLS guidelines	https://csrc.nist.gov/pubs/sp/800/52/r2/final
32	NIST FIPS 203 — ML-KEM	https://csrc.nist.gov/pubs/fips/203/final
33	NIST FIPS 204 — ML-DSA	https://csrc.nist.gov/pubs/fips/204/final

Abbreviations: **CCADB** = Common Certificate Authority Database · **ICANN** = Internet Corporation for Assigned Names and Numbers · **ITSP.40.062** = Information Technology Security Practitioner publication 40.062 · **ITSP.40.111** = Information Technology Security Practitioner publication 40.111 · **ITSM.40.001** = Information Technology Security Management publication 40.001 · **NIST** = National Institute of Standards and Technology · **SP** = Special Publication · **TLS** = Transport Layer Security · **FIPS** = Federal Information Processing Standard · **ML-KEM** = Module-Lattice-Based Key-Encapsulation Mechanism · **ML-DSA** = Module-Lattice-Based Digital Signature Algorithm

All links (3/4)

 Provided for convenience; a link is not an endorsement. Before opening any link, verify it with the VirusTotal **URL** (Uniform Resource Locator) scanner (Legal Notice §10).

#	Organization or document	Official address (click to open)
34	NIST FIPS 205 — SLH-DSA	https://csrc.nist.gov/pubs/fips/205/final
35	NIST Post-Quantum Cryptography project	https://csrc.nist.gov/projects/post-quantum-cryptography
36	NIST IR 8547 (initial public draft)	https://csrc.nist.gov/pubs/ir/8547/ipd
37	Cryptographic Module Validation Program	https://csrc.nist.gov/projects/cryptographic-module-validation-program
38	OpenSSL project	https://www.openssl.org/
39	OpenSSH	https://www.openssh.org/
40	OpenSSH 10.0 release notes	https://www.openssh.org/txt/release-10.0
41	WireGuard	https://www.wireguard.com/
42	WireGuard — protocol and cryptography	https://www.wireguard.com/protocol/
43	pfSense documentation — Certificate Management	https://docs.netgate.com/pfsense/en/latest/certificates/index.html
44	pfSense documentation — WireGuard	https://docs.netgate.com/pfsense/en/latest/vpn/wireguard/index.html
45	pfSense	https://www.pfsense.org/
46	OPNsense	https://opnsense.org/
47	CIS Critical Security Controls v8.1	https://www.cisecurity.org/controls/cis-controls-list
48	OWASP — TLS Cheat Sheet	https://cheatsheetseries.owasp.org/cheatsheets/Transport_Layer_Security_Cheat_Sheet.html
49	Mozilla SSL Configuration Generator	https://ssl-config.mozilla.org/
50	VirusTotal URL scanner	https://www.virustotal.com/gui/home/url

Abbreviations: **NIST** = National Institute of Standards and Technology · **FIPS** = Federal Information Processing Standard · **SLH-DSA** = Stateless Hash-Based Digital Signature Algorithm · **IR** = Interagency or Internal Report · **CIS** = Center for Internet Security · **OWASP** = Open Worldwide Application Security Project · **TLS** = Transport Layer Security · **SSL** = Secure Sockets Layer · **URL** = Uniform Resource Locator



All links (4/4)



Provided for convenience; a link is not an endorsement. Before opening any link, verify it with the VirusTotal **URL** (Uniform Resource Locator) scanner (Legal Notice §10).

#	Organization or document	Official address (click to open)
51	Professor Messer — Public Key Infrastructure	https://www.youtube.com/watch?v=xHAMEF7-inQ
52	Computerphile — Public Key Cryptography	https://www.youtube.com/watch?v=GSIDS_lvRv4
53	ByteByteGo — SSL , TLS , HTTPS Explained	https://www.youtube.com/watch?v=j9QmMEWmcfo
54	Computerphile — How Secure Shell Works	https://www.youtube.com/watch?v=ORcvSkgdA58
55	Professor Messer — Certificates	https://www.youtube.com/watch?v=cla94BZH_9s
56	3Blue1Brown — How secure is 256 bit security?	https://www.youtube.com/watch?v=S9JGmA5_unY
57	Computerphile — Elliptic Curves	https://www.youtube.com/watch?v=NF1pwjL9-DE
58	Criminal Code of Canada, s. 342.1	https://laws-lois.justice.gc.ca/eng/acts/c-46/section-342.1.html
59	Criminal Code of Canada, s. 342.2	https://laws-lois.justice.gc.ca/eng/acts/c-46/section-342.2.html
60	Criminal Code of Canada, s. 430 and 430(1.1)	https://laws-lois.justice.gc.ca/eng/acts/c-46/section-430.html
61	Québec — private-sector privacy act, Law 25	https://www.legisquebec.gouv.qc.ca/en/document/cs/P-39.1
62	Commission d'accès à l'information du Québec	https://www.cai.gouv.qc.ca/
63	Privacy Commissioner of Canada — PIPEDA	https://www.priv.gc.ca/en/privacy-topics/privacy-laws-in-canada/the-personal-information-protection-and-electronic-documents-act-pipeda/
64	EUR-Lex — GDPR , Regulation 2016/679	https://eur-lex.europa.eu/eli/reg/2016/679/oj
65	18 U.S. Code § 1030 — CFAA	https://www.law.cornell.edu/uscode/text/18/1030
66	EUR-Lex — Directive 2013/40/EU	https://eur-lex.europa.eu/eli/dir/2013/40/oj
67	BOE — Ley Orgánica 10/1995, Spanish Criminal Code	https://www.boe.es/buscar/act.php?id=BOE-A-1995-25444

Abbreviations: **SSL** = Secure Sockets Layer · **TLS** = Transport Layer Security · **HTTPS** = Hypertext Transfer Protocol Secure · **PIPEDA** = Personal Information Protection and Electronic Documents Act · **GDPR** = General Data Protection Regulation · **CFAA** = Computer Fraud and Abuse Act · **BOE** = Boletín Oficial del Estado, the Official State Gazette of Spain





Legal notice: what applies to this Material

§5 Your risk

You use the Material at your own risk, on your own equipment and time; Cyber.SoHo and the instructor are not liable for damage, data loss, suspensions, costs or legal consequences.

§6 No warranty

Provided "as is" and "as available"; tools, prices, links and best practices change.

§7 No professional advice

Not legal advice and not security consulting; real infrastructures need qualified professionals.

§8 Intellectual property

© 2026 Cyber.SoHo, all rights reserved. Personal, non-transferable study license: no redistribution, re-publication, resale or public posting without written permission. Your own lab work is yours.

§9 No affiliation

OpenSSL, OpenSSH, WireGuard, pfSense, OPNsense, Let's Encrypt, Chrome, Firefox, Windows, YouTube, TikTok and all other names are trademarks of their owners, used for identification only.

§11 Ethics and AI rule

White-hat only; report an accidental real vulnerability to its owner and the instructor. Any **AI** (Artificial Intelligence) use in a submission names the **LLM** (Large Language Model) and the exact prompts. The model drafts. The analyst decides. The analyst's name goes on the record.

§13 Indemnification

You hold Cyber.SoHo and the instructor harmless from claims caused by your own actions or violations.

§15 Governing law

Laws of the Province of Quebec, Canada; an invalid clause leaves the others in force; Version 1.0 replaces all previous versions.

Laws: Criminal Code of Canada s. 342.1, 342.2, 430(1.1) · Québec Law 25 · **PIPEDA** (Personal Information Protection and Electronic Documents Act) · **GDPR** (General Data Protection Regulation) · **EU** (European Union) Directive 2013/40/EU · United States **CFAA** (Computer Fraud and Abuse Act), **18 U.S.C.** (United States Code) §1030 · Spain, Ley Orgánica 10/1995



Legal Notice, Disclaimer, and Acceptable Use Policy — Cyber Defense (420-D03-AB) — Version 1.0 — Cyber.SoHo Educational Hub © 2026 | From the Industry to the Classroom — Building the IT's Future, Today and Together !
ALL Rights Reserved

